



Name Mustafa kamal

Reg NO sp23-bse-067

Subject SDA

LAB ASSIGMENT 1

USE CASE CODE

USECASE REGISTER

Description

This project implements a **user registration system** using a clean separation of concerns in **Node.js (OOP-based backend)** and a simple, attractive **HTML/CSS frontend**.

It allows users to input their **name, email, and password**, which is securely hashed using **bcrypt**.

The system checks for **duplicate emails** before saving a new user in an in-memory database.

The backend uses **Express.js** with modular controllers, services, and routes. Frontend submits data using **fetch API** to a /register endpoint.

The code structure follows best practices for **maintainability and scalability**.

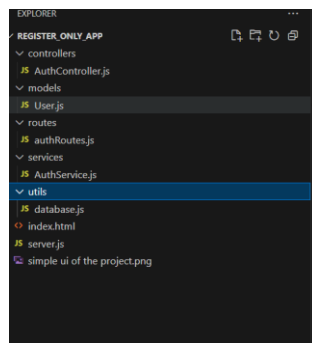
Frontened

```
index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Register</title>
6   <style>
7     body {
8       font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
9       background: linear-gradient(to right, #4facfe, #00f2fe);
10      display: flex;
11      height: 100vh;
12      justify-content: center;
13      align-items: center;
14    }
15    form {
16      background: #fff;
17      padding: 30px 40px;
18      border-radius: 15px;
19      box-shadow: 0 10px 25px #0000000a;
20      width: 350px;
21      text-align: center;
22    }
23    h2 {
24      color: #333;
25      margin-bottom: 20px;
26    }
27    input {
28      width: 100%;
29      padding: 12px;
30      margin: 10px 0;
31      border: 1px solid #ccc;
32      border-radius: 5px;
33    }
34    button {
35      width: 100%;
36      padding: 12px;
37      background-color: #007bff;
38    }
39  </style>
40 </head>
41 <body>
```

```
<html lang="en">
<head>
  <style>
    button {
      border-radius: 5px;
      cursor: pointer;
      font-size: 16px;
    }
    button:hover {
      background-color: #0056b3;
    }
  </style>
</head>
<body>
  <form id="register-form">
    <h2>Register</h2>
    <input type="text" id="reg-name" placeholder="Name" required />
    <input type="email" id="reg-email" placeholder="Email" required />
    <input type="password" id="reg-password" placeholder="Password" required />
    <button type="submit">Register</button>
  </form>
  <script>
    document.getElementById('register-form').addEventListener('submit', async (e) => {
      e.preventDefault();
      const name = document.getElementById('reg-name').value;
      const email = document.getElementById('reg-email').value;
      const password = document.getElementById('reg-password').value;

      const res = await fetch('http://localhost:3000/api/register', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name, email, password })
      });

      const data = await res.json();
      alert(data.message);
    });
  </script>
</body>
```



BACKENDED CODE

Authcontroller.js

```
JS AuthController.js X
controllers > JS AuthController.js > ...
1  const authService = require('../services/AuthService');
2
3  class AuthController {
4    register(req, res) {
5      const { name, email, password } = req.body;
6      const result = authService.register(name, email, password);
7      res.json(result);
8    }
9  }
10
11  module.exports = new AuthController();
```

Models

User

```
JS User.js X
models > JS User.js > ...
1  const bcrypt = require('bcryptjs');
2
3  class User {
4    constructor(name, email, password) {
5      this.name = name;
6      this.email = email;
7      this.passwordHash = bcrypt.hashSync(password, 8);
8    }
9  }
10
11  module.exports = User;
```

Routes

authRoutes.js

```
JS authRoutes.js X
routes > JS authRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const authController = require('../controllers/AuthController');
4
5  router.post('/register', (req, res) => authController.register(req, res));
6
7  module.exports = router;
```

Services

authRoutes.js

```
JS AuthService.js X
services > JS AuthService.js > AuthService > register
1  const db = require('../utils/database');
2  const User = require('../models/User');
3
4  class AuthService {
5    register(name, email, password) {
6      if (db.findUserByEmail(email)) {
7        return { success: false, message: 'Email already exists.' };
8      }
9
10     const user = new User(name, email, password);
11     db.addUser(user);
12     return { success: true, message: 'Registration successful!' };
13   }
14 }
15
16 module.exports = new AuthService();
```

database

```
JS database.js X
utils > JS database.js > ...
1  class Database {
2    constructor() {
3      this.users = [];
4    }
5
6    addUser(user) {
7      this.users.push(user);
8    }
9
10   findUserByEmail(email) {
11     return this.users.find((u) => u.email === email);
12   }
13 }
14
15 module.exports = new Database();
```

Frontend remaining part

```
JS server.js X
JS server.js > ...
1  const express = require('express');
2  const bodyParser = require('body-parser');
3  const cors = require('cors');
4  const authRoutes = require('./routes/authRoutes');
5
6  const app = express();
7  const PORT = 3000;
8
9  app.use(cors());
10 app.use(bodyParser.json());
11 app.use('/api', authRoutes);
12
13 app.listen(PORT, () => {
14   console.log(`Server running at http://localhost:${PORT}`);
15 });
```

SIMPLE UI INTERFACE FOR THE REGISTER

Register Form
Name
Email
Password